OpenDesign Vendoring Plan — Helix OTA frontends

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z **Authority:** consumer-side research/plan (§11.4.35). Does NOT modify any frontend or the token package — read-only analysis + ready-to-execute steps for two follow-up implementers. **Scope:** vendor `design-systems/helix-ota/{tokens.css,tailwind-v4.css}` into BOTH `clients/ota-manager/` and `dashboard/`; fix the ota-manager theme-toggle bug; add dark mode to the dashboard; re-prove both under the §11.4.170 host-render harnesses.

> §11.4.6 honesty note: every file path + line is cited from the tree as read on 2026-07-09. Items I could not prove statically are marked `UNCONFIRMED:` with the exact verification step the implementer must run.

---

0. The token package being vendored (facts)

`design-systems/helix-ota/tokens.css` (185 lines) ships OpenDesign **semantic** token names as **hex**, light + dark first-class:

- Base `:root` = **LIGHT** (full TOKEN_SCHEMA set).
- DARK re-bound three ways (tokens.css:159–184): `@media (prefers-color-scheme: dark) :root:not([data-theme="light"])`, `:root[data-theme="dark"]`, and `.dark`.
- Names: `--bg --surface --surface-warm --fg --fg-2 --muted --meta --border --border-soft --accent --accent-on --accent-hover --accent-active --success --warn --danger` + type/space/radius/elevation/motion/layout tokens.

`design-systems/helix-ota/tailwind-v4.css` maps those into a Tailwind-v4 `@theme` block (`--color-accent: var(--accent)` … + `@import "tailwindcss"`).

Both files are pure CSS — `grep -icE "function|=>|require\("` returns `0` for each. → they carry **zero JS/React dependency** (load-bearing for the React-18-vs-19 risk below).

---

1. Frontend A — `clients/ota-manager/` (React 19 + Vite 6 + Tailwind v4)

1.1 Current state (facts)

- Tailwind v4 setup: `vite.config.ts:7` uses `@tailwindcss/vite`; `src/index.css:1` = `@import "tailwindcss";`. `darkMode:"class"` + shadcn color mappings live in `tailwind.config.ts` (v3-style config). `postcss.config.js` uses `@tailwindcss/postcss`.
- Token model in `src/index.css`: shadcn **HSL-triple** vars consumed as `hsl(var(--X))`. Inverted theme contract:
  - base `:root` (index.css:13–40) = **DARK** palette.
  - `.light` (index.css:42–68) = **LIGHT** palette.
  - `@custom-variant dark (&:is(.dark *))` (index.css:3) → Tailwind `dark:` utilities need a `.dark` ancestor.
  - `* { border-color: hsl(var(--border)); }` (index.css:70–72) — every border reads `--border`.
- Theme store: `src/stores/ui-store.ts` — `theme:"dark"` default, `setTheme`/`toggleTheme` mutate zustand state + persist (`helix-ota-ui`). **Nothing writes the DOM.**
- Toggle UI: `src/features/layout/topbar.tsx:33-39` — `onClick={toggleTheme}` only swaps the `Sun`/`Moon` icon.

1.2 THE BUG (confirmed fact, §11.4.6)

`grep -rn "classList|documentElement|data-theme" clients/ota-manager/src/` → **no match** (only the CSS `.light` / `@custom-variant dark`). The `ui-store.theme` value is **never** written to a DOM class. Because the palette is switched by the presence of `.light` on the root and nothing ever adds it, the base `:root` (**DARK**) palette applies permanently; `toggleTheme` flips the store value + the icon but the pixels never change. This is the §11.4.170 forensic class (a control that looks toggled while the rendered surface stays broken).

1.3 Token-name COLLISION (the central reconciliation fact)

Three names exist in BOTH token sets with **incompatible value forms**:

name	index.css (shadcn, HSL, consumed `hsl(var())`)	helix-ota tokens.css (hex)
`--accent`	`217.2 32.6% 17.5%` (index.css:26)	`#2563eb` (tokens.css:64)
`--muted`	`217.2 32.6% 17.5%` (index.css:24)	`#64748b` (tokens.css:49)
`--border`	`217.2 32.6% 17.5%` (index.css:30)	`#e2e8f0` (tokens.css:55)

If helix-ota's hex `--border` wins the cascade, `hsl(var(--border))` becomes `hsl(#e2e8f0)` → **invalid** → **every border breaks**. This is why an unscoped import into the same `:root` is forbidden. (The non-colliding tokens — `--bg --fg --surface --surface-warm --accent-on --success --warn --danger` etc. — are safe.)

Cross-check: the light `--border` in both systems is the **same colour** (`214.3 31.8% 91.4%` == `#e2e8f0`); the collision is a *value-form* clash (HSL vs hex), not a palette disagreement — expected, since the package was derived from index.css.

1.4 RECOMMENDATION: LAYER (coexist), do NOT replace

REPLACE (rip out shadcn HSL vars) has an unbounded blast radius — every shadcn component + `tailwind.config.ts` reads `bg-background`, `text-muted-foreground`, `border`, `bg-primary`, …. LAYER keeps index.css authoritative for the existing stack and adds the OpenDesign semantic tokens alongside, with import order resolving the 3 collisions.

Do NOT import `tailwind-v4.css` into ota-manager: its `@theme` emits `--color-accent` / `--color-border` / `--color-muted` Tailwind utilities that collide with the same-named shadcn utilities already defined in `tailwind.config.ts`. Vendor only `tokens.css` (the CSS-variable layer); keep `tailwind-v4.css` in the package as reference.

1.5 Exact steps (ota-manager)

1. **Copy** `design-systems/helix-ota/tokens.css → clients/ota-manager/src/styles/opendesign-tokens.css` (preserve the provenance header comment verbatim; no value edits).

2. **Wire it collision-safely** in `src/index.css` — add the import immediately **after** line 1 (`@import "tailwindcss";`) and **before** the shadcn `:root {` block (index.css:13). Because the shadcn `:root` block is later in source order at equal specificity, its HSL `--accent/--muted/--border` win the 3 collisions at runtime, while all non-colliding OpenDesign tokens remain live:

```
@import "tailwindcss";
@import "./styles/opendesign-
        tokens.css";   /* NEW — OpenDesign semantic layer */
@custom-variant dark (&:is(.dark *));
/* …existing shadcn :root / .light blocks unchanged (HSL wins
        the 3 collisions)… */
```

   (No change to `main.tsx` needed — index.css is already imported there at main.tsx:7, and the visual harness re-imports index.css, so the harness stays in sync automatically — see 1.7.)

3. **Theme-invariance is preserved:** the two systems flip in *opposite* directions relative to base `:root` (index.css base=dark, `.light`=light; opendesign base=light, `.dark`=dark). Applying **exactly one of `.light`/`.dark`** matching the theme makes BOTH agree (light → `.light` present: shadcn light ✓, opendesign has no `.dark` so base=light ✓; dark → `.dark` present: shadcn base=dark + `dark:` variant active ✓, opendesign `.dark`=dark ✓). The toggle-fix (1.6) MUST therefore add `.dark` in dark mode too, not merely remove `.light`.

1.6 THE TOGGLE-FIX — exact edit

File: `clients/ota-manager/src/stores/ui-store.ts`

Add a module-level DOM applier (mirrors the harness contract at `visual/harness.tsx:32-34`) and call it from `setTheme`, `toggleTheme`, and on persist rehydrate:

```
// NEW — apply the theme to the DOM (the missing wiring). Mirrors
        visual/harness.tsx.
function applyThemeClass(theme: Theme) {
  const el = document.documentElement;
  el.classList.remove("light", "dark");
```

```
  el.classList.add(theme);            // exactly one
        of .light / .dark
  el.setAttribute("data-theme", theme); // optional; also
        satisfies opendesign :root[data-theme="dark"]
}
```

Change the two mutators:

```
setTheme: (theme) => { applyThemeClass(theme); set({ theme }); },

toggleTheme: () =>
  set((state) => {
    const theme: Theme = state.theme === "dark" ? "light" :
        "dark";
    applyThemeClass(theme);
    return { theme };
  }),
```

Apply the **persisted** value on rehydrate (persist config, ui-store.ts:34–41):

```
{
  name: "helix-ota-ui",
  partialize: (state) => ({ sidebarCollapsed:
        state.sidebarCollapsed, theme: state.theme }),
  onRehydrateStorage: () => (state) => { if (state)
        applyThemeClass(state.theme); }, // NEW
}
```

Belt-and-suspenders (avoid first-paint FOUC before rehydrate): append to `src/main.tsx` after the `import "./index.css";` line — `import { useUiStore } from "@/stores/ui-store"; applyThemeClass(useUiStore.getState().theme);` — OR expose `applyThemeClass` and call `useUiStore.getState()` there. (Optional; `onRehydrateStorage` already covers the persisted case.)

No `topbar.tsx` change is required — it already calls `toggleTheme`; the fix is entirely in the store. `document` always exists (Vite SPA / Tauri / jsdom tests).

1.7 W2 harness re-proof (`clients/ota-manager/visual/`)

- The harness (`visual/harness.tsx:30-34`) applies `.light`/`.dark` itself from `?theme=`, independent of `ui-store`, and `visual/harness.css:5` re-imports `../src/index.css` — so the vendored `opendesign-tokens.css` is **automatically** in the harness render once step 1.5.2 lands. Re-run:

```
cd clients/ota-manager && pnpm hostrender      #
        package.json:17
```

It regenerates baseline/rerender/mutated PNGs + `results.json` under `docs/qa/20260709-ota-manager-hostrender/` and runs the dual oracle (image-diff + OCR/layout) with §11.4.107(10) self-validation for **both** themes. Expected: PASS, proving the OpenDesign layer renders correctly light+dark.

- **Pixel note (§11.4.170):** LoginPage renders with the shadcn HSL palette; since the 3 collided tokens keep their HSL values and no component is repointed to OpenDesign tokens in this increment, LoginPage pixels should be **unchanged** → baselines likely stable. If any baseline differs, **regenerate** the golden PNGs (pixels changed ⇒ new goldens are the evidence) and re-run the dual oracle.

- The harness bypasses `ui-store`, so it does NOT prove the *store wiring*. Add the toggle-fix's §11.4.115 RED→GREEN guard as a jsdom unit test (e.g. `src/stores/ui-store.test.ts`): assert that before the fix `toggleTheme()` leaves `document.documentElement.classList` unchanged (RED on the pre-fix store) and after the fix it flips `.dark` ↔ `.light` (+ `data-theme`). This is the anti-bluff proof that the toggle now moves the DOM.

---

2. Frontend B — `dashboard/` (React 18 + Vite 5, no Tailwind, light-only)

2.1 Current state (facts)

- No Tailwind, no global CSS, no `src/*.css`, no CSS import in `src/main.tsx`. All styling is **inline `style` objects with hardcoded hex**:
  - `src/components/ui.tsx:151-220` — `styles` + `badgeTone` (Card/Button/Field/TextInput/Badge/ProgressBar/Table/ErrorPanel/EmptyState).
  - `src/components/AppShell.tsx:82-114` — shell/header/nav/logout.
  - `src/screens/LoginScreen.tsx:52` — inline `#854d0e` notice.
- 8 screens routed in `src/App.tsx:29-71` (Overview, ArtifactUpload, Releases, Deployments, Fleet, Groups, Audit, Login) — all consume the `ui.tsx` primitives.
- No dark mode anywhere. W3 harness (`hostrender/login.hostrender.spec.ts:16-19`) states **LIGHT ONLY** explicitly; one committed baseline `login-card-light-chromium-linux.png`.

2.2 RECOMMENDATION

Vendor `tokens.css` **only** (plain CSS custom properties — dashboard has no Tailwind, so `tailwind-v4.css` is inapplicable; leave it in the package). Map the inline hex to `var(--token)` and add a real theme toggle that flips the `<html>` class/`data-theme` — tokens.css already ships `.dark` + `:root[data-theme="dark"]` so `var(--token)` reads flip with zero per-component logic.

2.3 Exact steps (dashboard)

1. **Copy** `design-systems/helix-ota/tokens.css` → `dashboard/src/styles/tokens.css`.
2. **Import once** at the top of `src/main.tsx` (before render): `import "./styles/tokens.css";`
3. **Map `ui.tsx` inline hex → tokens** (`src/components/ui.tsx:151–220`). Recommended mapping: | style | current | → token | |—|—|—| | `card.background` | `#fff` | `var(--surface)` | | `card.border` | `1px solid #e2e5ea` | `1px solid var(--border)` | | `card.borderRadius` | `8` | `var(--radius-md)` (12px, per DESIGN.md cards) | | `cardTitle` (color) | inherit | `var(--fg)` (set explicitly) | | `btn.borderRadius` | `6` | `var(--radius-sm)` | | `btnPrimary.background` | `#2563eb` | `var(--accent)` | | `btnPrimary.color` | `#fff` | `var(--accent-on)` | | `btnSecondary.background` | `#fff` | `var(--surface)` | | `btnSecondary.color` | `#1f2937` | `var(--fg)` | | `btnSecondary.borderColor` | `#cbd2dc` | `var(--border)` | | `fieldLabel.color` | `#4b5563` | `var(--muted)` | | `input.border` | `1px solid #cbd2dc` | `1px solid var(--border)` | | `input.background/color` | (browser default) | `var(--surface)` / `var(--fg)` (set explicitly so dark inputs are legible) | | `input.borderRadius` | `6` | `var(--radius-sm)` | | `progressTrack.background` | `#eef1f5` | `var(--surface-warm)` | | `progressFill.background` | `#2563eb` | `var(--accent)` | | `th.borderBottom` | `2px solid #e2e5ea` | `2px solid var(--border)` | | `th.color` | `#4b5563` | `var(--muted)` | | `errorPanel.background` | `#fef2f2` | `color-mix(in oklab, var(--danger), transparent 92%)` | | `errorPanel.border` | `1px solid #fca5a5` | `1px solid color-mix(in oklab, var(--danger), transparent 55%)` | | `errorPanel.color` | `#991b1b` | `var(--danger)` | | `empty.color` | `#6b7280` | `var(--muted)` | | `badgeTone.neutral` | `#eef1f5/#374151` | `var(--surface-warm)` / `var(--fg)` | | `badgeTone.ok` | `#dcfce7/#166534` | `color-mix(…var(--success)…)` / `var(--success)` | | `badgeTone.warn` | `#fef9c3/#854d0e` | `color-mix(…var(--warn)…)` / `var(--warn)` | | `badgeTone.err` | `#fee2e2/#991b1b` | `color-mix(…var(--danger)…)` / `var(--danger)` | |

`badgeTone.info` | `#dbeafe/#1e40af` | `color-mix(…var(--accent)…)` / `var(--accent)` |

4. **Map `AppShell.tsx` inline hex → tokens** (`src/components/AppShell.tsx:82-114`): `shell.background #f5f7fa → var(--bg)`; `shell.color #111827 → var(--fg)`; content-legibility tokens as needed. The dark navy header (`header.background #0f172a`, `#fff` text, nav/logout slate hexes) is a fixed brand chrome bar — **decision point:** keep it fixed (brand) OR map to `var(--surface-warm)`/`var(--fg)`/`var(--border)` so it also inverts. Recommend keeping the header a fixed brand bar and toning only its border via `var(--border)` (least visual churn); the **toggle button** (step 5) lives in this header's `user` cluster.
5. **Map `LoginScreen.tsx`** notice `#854d0e → var(--warn)` (LoginScreen.tsx:52).
6. **ADD dark mode:**

   - New file `dashboard/src/theme.ts`:

```ts
export type Theme = "light" | "dark";
export function applyTheme(t: Theme) {
  document.documentElement.setAttribute("data-theme",
        t); // authoritative over prefers-color-scheme
  try { localStorage.setItem("helix-ota-dash-theme", t); }
        catch {}
}
export function initTheme(): Theme {
  let t: Theme | null = null;
  try { t = localStorage.getItem("helix-ota-dash-theme")
        as Theme | null; } catch {}
  if (!t) t = window.matchMedia?.("(prefers-color-scheme:
        dark)").matches ? "dark" : "light";
  applyTheme(t);
  return t;
}
export function toggleTheme(cur: Theme): Theme { const n:
        Theme = cur === "dark" ? "light" : "dark";
        applyTheme(n); return n; }
```

   - In `src/main.tsx`, call `initTheme()` **before** `createRoot(...).render(...)` (and after the tokens.css import).

   - In `AppShell.tsx`, add a `useState<Theme>` seeded from `localStorage`/current `data-theme`, and a toggle button in the `styles.user` cluster (AppShell.tsx:66–73) calling `setTheme(toggleTheme(theme))`. Because every primitive now reads `var(--token)`, the flip is automatic — no per-screen edits.

- Use `data-theme="light|dark"` **explicitly** (not bare `.dark`) so it overrides the `@media (prefers-color-scheme: dark)` block in tokens.css:159 (see Risk R4).

2.4 W3 harness re-proof (`dashboard/hostrender/`)

- Parametrize `hostrender/login.hostrender.spec.ts` over `["light","dark"]`: before each screenshot set the theme deterministically — either `await page.evaluate(t => document.documentElement.setAttribute("data-theme", t), theme)` OR click the real toggle (the latter also proves the toggle) — and emit `login-card-light.png` **and** `login-card-dark.png`. Keep `injectRegression` + the pixelmatch/`toHaveScreenshot` self-validation (golden-good + golden-bad, §11.4.107(10)) for **both** themes. Remove the "LIGHT ONLY / dark not implemented" note (spec:16–19) once dark lands.
- Regenerate baselines: `npx playwright test --config=playwright.hostrender.config.ts --update-snapshots`. The existing **light** baseline WILL change — `card.border #e2e5ea → var(--border) = #e2e8f0` (a real 3-hex delta) — so the light golden regenerates too (§11.4.170: pixels changed ⇒ new goldens are the evidence), plus the new dark golden.

---

3. Risks (§11.4.92 blast-radius)

R1 — React 18 (dashboard) vs 19 (ota-manager). CONFIRMED non-issue. The vendored tokens are **pure CSS** (`grep -icE "function|=>|require\(" … = 0` for both `tokens.css` and `tailwind-v4.css`) — no JS, no npm dep, no bundler/runtime coupling to React. The only JS added is the theme helpers, which use plain DOM API (`classList`/`setAttribute`/`localStorage`) and are React-version-agnostic. No `package.json` dependency is added to either frontend.

R2 — Token-name collisions (ota-manager only). `--accent`, `--muted`, `--border` exist as HSL (shadcn) AND hex (helix-ota). Unscoped import → invalid `hsl(#hex)` → borders/accents break. **Resolved** by import order (opendesign layer imported before the shadcn `:root` block so HSL wins the 3 names — 1.5.2) and by **not** importing `tailwind-v4.css` into ota-manager (its `@theme --color-accent/-border/-muted` would collide with the shadcn Tailwind utilities). Dashboard has **no** prior CSS vars → no name collision there.

R3 — §11.4.170 baseline regeneration. Vendoring changes pixels ⇒ goldens regenerate. ota-manager: LoginPage likely **unchanged** (HSL palette retained; no component repointed) — regenerate only if a diff appears. dashboard: light golden **changes** (border `#e2e5ea` → `#e2e8f0`) + a new dark golden is added. In both cases regenerate the goldens, re-run the dual oracle (image-diff + OCR/layout) with its golden-good/golden-bad self-validation, and commit the new goldens as the §11.4.170 evidence. Regeneration is expected and compliant, not a defect.

R4 — dashboard `prefers-color-scheme` auto-dark behavior change. Once `tokens.css` is imported, its `@media (prefers-color-scheme: dark) :root:not([data-theme="light"])` block (tokens.css:159–171) makes the dashboard **auto-dark on a dark-preference OS even before a toggle is wired**. Mitigation: `initTheme()`/`applyTheme()` always set an explicit `data-theme` on `<html>`, which takes the `:root[data-theme="…"]` path and neutralizes the media query. Land the toggle + `initTheme()` in the **same** change as the tokens import so no window of uncontrolled auto-dark ships.

R5 — `UNCONFIRMED`: Tailwind v4 config-loading mechanism (ota-manager). `src/index.css` has **no** `@config`/`@theme`/`@source` (grep confirmed), yet the shadcn utilities (`bg-background`, `text-muted-foreground`, …) in `tailwind.config.ts` clearly render. The exact mechanism (v4 back-compat auto-detection of a root `tailwind.config.ts` by `@tailwindcss/vite`) is not provable by static read. **Verification step:** run `cd clients/ota-manager && pnpm build` (or `pnpm hostrender`) after step 1.5 and confirm the utilities still emit and the added `opendesign-tokens.css` variables resolve. This does **not** block the plan (the vendored layer is CSS-variable only; it adds no `@theme`), but the implementer must confirm before considering step 1.5 done. It is the reason 1.4 recommends NOT importing `tailwind-v4.css` here.

R6 — theme-contract inversion (ota-manager). index.css base `:root`=DARK / `.light`=light; opendesign base `:root`=LIGHT / `.dark`=dark. The toggle-fix MUST add **exactly one of `.light`/`.dark`** (add `.dark` in dark mode, not just remove `.light`) so both systems agree AND the `@custom-variant dark (&:is(.dark *))` utilities activate. The 1.6 `applyThemeClass` does exactly this (mirrors `visual/harness.tsx:32–34`).

---

4. Execution order (both implementers)

1. ota-manager: 1.5 (vendor+wire) → 1.6 (toggle-fix + unit guard) → 1.7 (re-run `pnpm hostrender`, regenerate goldens if diff, verify R5).
2. dashboard: 2.3 steps 1–5 (vendor + map inline styles) → 2.3 step 6 + R4 (add dark, land with tokens import atomically) → 2.4 (parametrize W3 spec, `--update-snapshots`).
3. Each closure carries its §11.4.170 host-render evidence dir + the golden-good/golden-bad self-validation verdict.

Sources verified

- Files read 2026-07-09 from the working tree (paths/lines cited inline). No external service docs required (pure in-repo CSS/React vendoring).